

OGARD

Synthetic Benchmark Methodology and Validation Plan

Controlled evaluation of identity reconciliation, review routing, contradiction detection, and auditability on synthetic grid-asset evidence

Deniz Bektas

Document status and use

Field	Value
Document	OGARD Synthetic Benchmark Methodology and Validation Plan
Version	Public edition v0.1.1
Date	Original draft: August 2026; public edition: 24 September 2026
Author	Deniz Bektas
Public release status	Public — published in the OGARD GitHub repository
Results status	No executed or independently reproduced results are reported
Related documents	OGARD Technical Whitepaper; OGARD Implementation Guide; OGARD Architecture and Data Model Specification. Public editions v0.1.1; companion technical content: August 2026.
Scope	Synthetic evaluation of selected OGARD reconciliation, review, contradiction, and audit functions
Repository	https://github.com/DenBek/ogard
Documentation license	Creative Commons Attribution 4.0 International (CC BY 4.0)
Reference implementation license	Apache License 2.0 for original repository code; see LICENSE.md.

Results boundary. This document defines how a future OGARD v0.1 benchmark should be constructed, run, evaluated, independently reproduced, and reported. It must not be cited as evidence that any benchmark result has already been achieved. Separate executable results: the selected five-scenario, six-decision evaluation first released in 0.2.0 is available at <https://github.com/DenBek/ogard/tree/fa84c5026ca1afcb74345a08e2be32db7122af8f/demonstrator/results>. It is narrower than this full benchmark plan.

Independence and confidentiality. The benchmark design is intended for original synthetic or public data only. It must contain no employer source code, client data, non-public utility records, proprietary product material, or operator-specific implementation detail.

Normative language. MUST identifies a requirement for a benchmark result to be represented as reproducible under this methodology. SHOULD identifies a recommended practice that may be varied with documented rationale. MAY identifies an optional extension.

Abstract

This document specifies a controlled synthetic benchmark for OGARD, the Open Grid Asset Reliability Data Framework. The benchmark is designed to test whether selected reference methods can reconcile clear synthetic identity relationships, rank or route ambiguous cases without creating false certainty, surface seeded contradictions, preserve unresolved cases, and produce a complete audit trail. It defines evaluation questions, synthetic-estate requirements, scenario classes, ground-truth separation, baseline configurations, record-lifecycle accounting, metrics, experimental procedure, release gates, artifact requirements, and independent reproduction. It deliberately reports no results. A separate results report may be published only after the executable artifacts exist, all metrics can be recomputed, record totals reconcile without overlap, and an independent reviewer has completed a clean-environment reproduction.

Contents

1. Purpose, scope, and claims boundary
 2. Evaluation questions
 3. Benchmark design and synthetic estate
 4. Scenario catalogue and ground truth
 5. Evaluation configurations and baselines
 6. Metrics and record accounting
 7. Experimental procedure
 8. Error analysis and sensitivity checks
 9. Reproducibility artifact package
 10. Independent review and reproduction
 11. Results-reporting requirements and release gate
 12. Limitations and non-claims
- References
- Appendix A. Minimum configuration record
- Appendix B. Results and disposition template
- Appendix C. Artifact checklist
- Change log

1. Purpose, scope, and claims boundary

The purpose of the benchmark is to characterize selected OGARD reference methods under controlled synthetic conditions. It is not intended to simulate the complete operational environment of an electricity-network operator. The benchmark focuses on data-engineering behavior that can be evaluated against known synthetic truth: identity reconciliation, uncertainty routing, contradiction detection, and decision traceability.

1.1. In scope

- Functional-location, equipment-unit, and telemetry-channel identity relationships.
- Deterministic mapping rules and documented similarity-based candidate scoring.
- Routing of ambiguous or insufficiently supported cases to review.
- Detection and preservation of seeded contradictions.
- Time-valid installation and identifier relationships.
- Completeness of mapping, review, contradiction, and audit records.

1.2. Out of scope

- Live utility performance, outage reduction, restoration-time improvement, or maintenance optimization.
- Production-scale throughput, cybersecurity validation, human-factors assessment, or operational usability.
- Autonomous fault diagnosis, failure prediction, or operational control.
- Formal IEC CIM conformance or certification.
- Executed large-language-model results in the v0.1 core benchmark.

Interpretation boundary. A strong synthetic result can show only that a defined method behaved as measured on defined synthetic data. It cannot establish performance, adoption, or reliability impact in an operator environment.

2. Evaluation questions

The v0.1 benchmark should answer a small set of explicit questions rather than optimize one headline score.

ID	Evaluation question	Primary evidence
Q1	Can clear synthetic relationships be accepted correctly through versioned deterministic rules?	Accepted mappings, ground truth, rule IDs, and audit records
Q2	Can similarity scoring improve candidate ranking while keeping unsupported cases out of automatic acceptance?	Candidate ranks, thresholds, accepted mappings, false accepts, and review routing
Q3	Are ambiguous, incomplete, or temporally conflicting cases routed to review or left unresolved rather than forced?	Review-routing labels, unresolved cases, and reason codes
Q4	Are seeded cross-source contradictions surfaced without silently overwriting the conflicting evidence?	Contradiction flags, false positives, detection times, and source references
Q5	Can every material outcome be reconstructed from the benchmark audit artifacts?	Audit completeness and record-lifecycle closure
Q6	What review workload does each configuration create, and is the queue feasible under a declared reviewer-capacity assumption?	Routed volume, review minutes or declared handling time, queue completion, unresolved volume, and capacity assumption

The benchmark should report trade-offs. Higher automatic coverage is not inherently better if it increases false acceptance. Review routing is an intended safety outcome when the available evidence does not support automatic resolution.

3. Benchmark design and synthetic estate

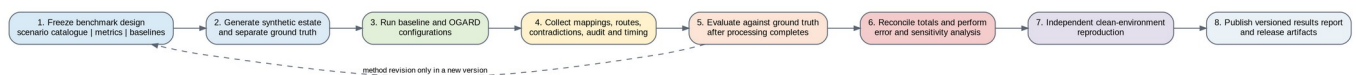


Figure 1. Required benchmark lifecycle from frozen design through independent reproduction.

The benchmark uses the core entity names, identifier regimes, and four logical processing layers defined in the companion OGARD Architecture and Data Model Specification. It evaluates selected reconciliation, review, contradiction, and audit behavior; it does not test derived asset-status rules.

3.1. Synthetic estate

The benchmark generator should create a configurable estate of canonical assets and source/evidence records. The initial public run need not be large; it must be large and varied enough to exercise every declared scenario class more than once and to support transparent error analysis. Estate size, source-record counts, asset-class composition, and scenario frequencies **MUST** be stored in a versioned configuration rather than embedded only in narrative text.

The core source classes should be limited to those used by the v0.1 architecture: asset-register records, maintenance or work-order records, telemetry-channel records, technical-document references, inspection or condition records, and fault or outage events. Additional classes may be added only if the benchmark report states how they affect the expected behavior.

3.2. Time and identity

The generator **MUST** distinguish functional location, physical equipment identity, and telemetry identity. It **SHOULD** include time-valid installation relationships so that replacement, relocation, tag drift, and late-arriving records can be evaluated without treating every identifier as permanently equivalent.

3.3. Determinism and versioning

A published run **MUST** record the model profile version, schema bundle version, generator version, source-code revision or release tag, random seed, configuration file, rule-library version, scoring configuration, threshold configuration, evaluation-script version, and execution environment. When the same deterministic configuration is rerun under the same conditions, the generated estate and reference outputs should be identical or differences must be explicitly documented.

4. Scenario catalogue and ground truth

4.1. Minimum scenario classes

Scenario class	What it tests	Expected safe behavior	Scenario rationale or provenance
Identifier drift	Aliases, abbreviations, punctuation, or changed naming conventions	Resolve only where evidence and thresholds support the relationship; otherwise review	Common integration failure caused by aliases, abbreviations, punctuation, and local naming changes.
Conflicting status	Different source systems report incompatible states	Preserve and surface a typed contradiction	Derived from cross-system status semantics and delayed source updates.
Stale telemetry	Expected telemetry is absent or outside a configured freshness window	Flag the evidence condition without independently declaring physical failure	Derived from historian freshness, tag retirement, and broken or delayed feeds.
Duplicate or competing records	Multiple records appear to represent the same or different assets	Avoid silent duplicate acceptance and preserve competing evidence	Derived from duplicate registration, merges, splits, and unresolved identity.
Missing physical identifier	Serial or equipment identity is absent	Use other evidence conservatively or route to review	Derived from incomplete serial history and functional-location-only records.
Late-arriving update	A record arrives after a replacement or status transition	Use event time and validity windows rather than arrival order alone	Derived from event-time versus arrival-time differences and retrospective corrections.
Mismatched cross-source reference	Work-order or telemetry reference points to the wrong asset context	Reject, review, or create a contradiction with traceable reason	Derived from work-order, tag, or document references attached to

Scenario class	What it tests	Expected safe behavior	Scenario rationale or provenance
Adverse condition evidence	Inspection or condition record contains an adverse signal	Attach to the correct asset context and preserve source evidence; do not autonomously diagnose	the wrong context. Derived from inspections or condition records that must be attached without autonomous diagnosis.
Correlated-source blind spot	Several apparently independent source records are internally consistent because they inherit the same incorrect upstream identity or configuration.	The method may accept the consistent relationship unless independent evidence or a contradiction is available; the limitation must be reported rather than treated as a successful detection.	Adversarial case added to demonstrate that cross-source agreement is not proof of physical truth and that the benchmark can contain known undetectable conditions.

4.2. Ground-truth separation

The synthetic generator MUST write ground truth to a separate artifact that is not read by the reconciliation process. Processing code may use only the synthetic source records and declared configuration. The evaluation code may access ground truth only after processing outputs have been finalized. This separation reduces the risk that benchmark logic is tuned to or directly reads the expected answer.

4.3. Calibration and evaluation partitions

If thresholds or similarity weights are calibrated using synthetic examples, the calibration cases MUST be separate from the evaluation cases. Scenario templates may be shared, but record instances, identifiers, timestamps, and seeded relationships used for final evaluation should not be used to set the reported thresholds. The results report must state whether a holdout partition was used and how it was generated.

4.4. Truth labels

Ground truth should identify, at minimum, the correct asset relationship or no-match outcome, whether the case is intended to require review, any seeded contradiction and its availability time, temporal validity, and the expected reason category. Cases intentionally designed to be unresolved should be labeled as such rather than counted as missed automatic matches.

4.5. Scenario provenance and catalogue governance

The v0.1 scenario catalogue is author-selected. Each scenario must therefore state the practical failure pattern it represents and the expected safe behavior. The catalogue should be reviewed by technical and utility-domain practitioners before a results release, and later versions should record whether a scenario was author-proposed, reviewer-proposed, or derived from an authorized public source.

The catalogue must include at least one case the method is not expected to detect reliably, such as several sources that agree because they inherit the same incorrect upstream identity. Such a case should be reported as a known blind spot rather than forced into a positive metric.

5. Evaluation configurations and baselines

The benchmark should compare the complete reference method with simple baselines. A result is easier to interpret when it shows what each added stage contributes.

Configuration	Included capability	Purpose
B0: exact-key baseline	Exact identifiers only; no alias rules, similarity, or review logic	Establish a simple lower-complexity comparison
B1: deterministic rules	Versioned exact, alias, time-window, and relationship rules	Measure the contribution of explicit rules
B2: deterministic plus similarity	B1 plus declared similarity methods and thresholds	Measure candidate-ranking and controlled auto-accept behavior
B3: full controlled workflow	B2 plus review routing, contradiction handling, and complete audit records	Evaluate the intended v0.1 OGARD control pattern

No live large-language model should be invoked in the v0.1 core benchmark. Records that might later be eligible for optional AI-assisted recommendation should be reported as review-routed or AI-eligible, not as AI-resolved.

5.1. Threshold declaration

Each threshold profile **MUST** be versioned and published with the run. The report should distinguish auto-accept, review, and reject bands. Thresholds must not be changed after evaluation results are inspected unless the run is relabeled as exploratory and a new version is executed on a new evaluation partition.

6. Metrics and record accounting

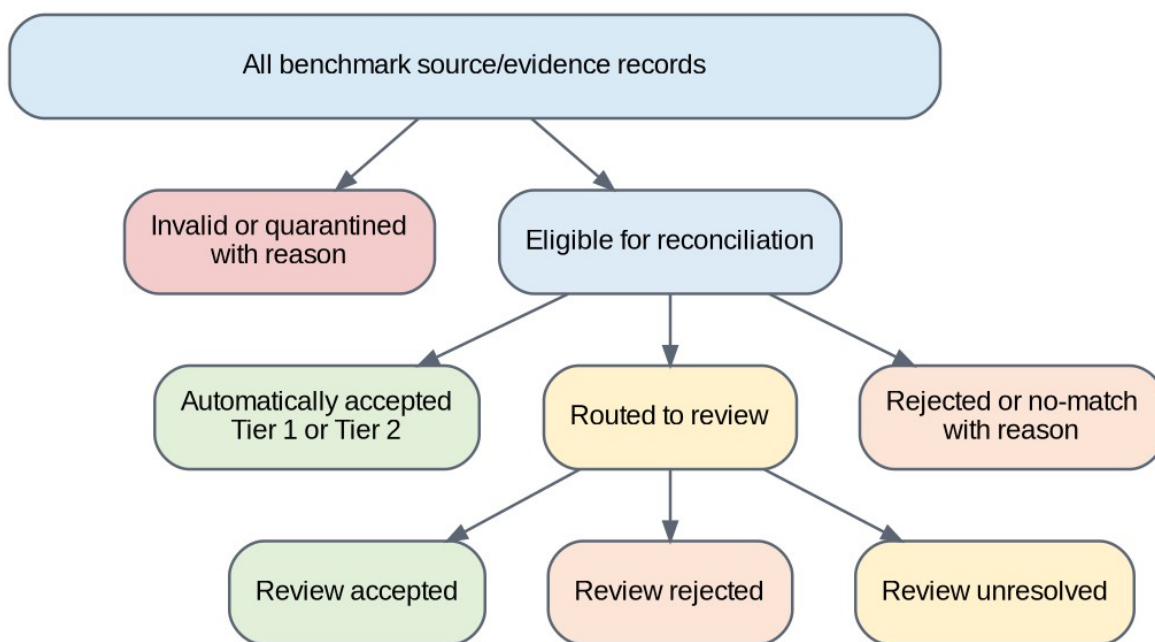


Figure 2. Mutually exclusive record dispositions required for lifecycle accounting.

6.1. Required mapping metrics

Metric	Definition	Reporting note
Accepted-mapping precision	Correct accepted mappings divided by all accepted mappings	Report separately for automatic and review-accepted outcomes
Accepted-mapping recall	Correct accepted mappings divided by all ground-truth mappings eligible for acceptance	Exclude cases explicitly designed to remain unresolved; report exclusions
Automatic coverage	Automatically accepted records divided by reconciliation-eligible records	Coverage is not a safety metric by itself

Metric	Definition	Reporting note
Unsafe auto-accept rate	Incorrect automatic accepts divided by all automatic accepts	Must be shown even when the numerator is zero
Review-routing recall	Cases labeled as requiring review that were actually routed to review	Report false review routing separately
Unresolved-review volume	Records still unresolved at run completion	Report count, share, and reason categories
Review burden	Review-routed records multiplied by measured or declared handling time, reported with queue completion and unresolved volume	Report by configuration and threshold profile; state the reviewer-capacity assumption and do not infer live operator staffing requirements

6.2. Contradiction and audit metrics

Metric	Definition
Contradiction precision	Correctly surfaced seeded contradictions divided by all surfaced contradiction cases
Contradiction recall	Correctly surfaced seeded contradictions divided by all seeded contradiction cases available to the method
Detection latency	Benchmark event, step, or batch interval from contradiction availability to surfaced flag
Audit completeness	Material outcomes containing every required decision and provenance field divided by all material outcomes
Lifecycle closure	All input records are accounted for exactly once in mutually exclusive terminal or active dispositions

6.3. Denominators and zero results

Every metric **MUST** define its denominator in the results file and report the underlying counts. Percentages without counts are insufficient. Zero false positives, perfect detection, or any other apparently perfect result must still be accompanied by the number and diversity of evaluated cases, error analysis, and limitations.

6.4. Record-lifecycle closure

The results report **MUST** include a reconciliation table proving that the total number of input records equals the sum of mutually exclusive dispositions. A reviewed record should not be counted both as a review-tier record and again as an additional unresolved tier. Review entry, review outcome, and final disposition are different lifecycle attributes and should be reported separately.

Review workload is a principal output, not a secondary diagnostic. Every primary and sensitivity configuration should report routed volume, assumed or measured minutes per review, estimated review hours, queue completion, and unresolved volume. These measurements characterize benchmark workflow burden only and do not establish staffing needs in a live operator environment.

7. Experimental procedure

1. Freeze the scenario catalogue, generator configuration, ground-truth schema, baseline configurations, metrics, and threshold profiles under a release-candidate tag.
2. Create a clean execution environment from the published environment specification. Record operating system, runtime version, dependencies, and relevant resource limits.
3. Generate the synthetic estate and separate ground-truth artifacts from the declared seed and configuration. Compute checksums for the generated files.
4. Run B0 through B3 without modifying source records, ground truth, rules, thresholds, or evaluation logic between configurations except as declared by the configuration itself.
5. Write machine-readable outputs for candidates, accepted and rejected mappings, review routing and outcomes, contradictions, audit events, timing, and errors.

6. After processing is complete, run the evaluation code against the frozen outputs and ground truth. Produce both machine-readable and human-readable result summaries.
7. Verify lifecycle closure, metric arithmetic, file checksums, and consistency between the report tables and result files.
8. Perform scenario-level error analysis and declared sensitivity checks. Record all deviations, failures, and manual interventions.
9. Package the artifacts and conduct a clean-environment independent reproduction before publishing a results report as reproduced.

7.1. Automation and logging

The run SHOULD be automated through one documented command or a short sequence of commands. Logs should identify successful stages, failed checks, optional dependencies, runtime, and output locations without requiring a reviewer to infer whether execution completed.

7.2. No undocumented intervention

Any manual correction, review action, data edit, or rerun must be logged. A result that depends on undocumented intervention must not be represented as reproducible. Synthetic human-review outcomes may be predeclared or executed through a documented review script, but the method must state which approach was used.

8. Error analysis and sensitivity checks

Aggregate metrics can conceal important failure modes. The results report should include error analysis sufficient to explain incorrect accepts, missed matches, false contradictions, missed contradictions, unnecessary review routing, and unresolved cases.

Required analysis	Minimum output
Scenario-level performance	Counts and metrics by scenario class and configuration
False-accept analysis	Source record, expected relationship, accepted relationship, rule or score, and cause
False-reject or missed-match analysis	Expected relationship, actual disposition, and missing or conflicting evidence
Review analysis	Review entries by reason, accepted/rejected/unresolved outcome, and queue completion
Threshold sensitivity	Results for at least one stricter and one looser declared threshold profile, or a documented reason not to run the check
Baseline comparison	Difference between B0, B1, B2, and B3 on the same frozen estate

The purpose of sensitivity analysis is not to select the best result after the fact. It is to show how coverage, false acceptance, and review burden change when thresholds change. The primary result must remain tied to the predeclared configuration.

9. Reproducibility artifact package

The benchmark should be packaged so that a technically qualified reviewer can obtain the release, create the environment, run the benchmark, and recompute the published tables without private assistance. The package should be documented, consistent with the associated report, complete enough to generate the reported results, and exercisable.

Artifact	Minimum requirement
README	Purpose, artifact inventory, environment, commands, expected runtime, outputs, limitations, and troubleshooting
Environment specification	Pinned dependencies or lock file; container definition where practical
Generator or frozen synthetic data	Versioned source records and deterministic generation instructions

Artifact	Minimum requirement
Ground truth	Separate file or deterministic output not read by processing code
Scenario catalogue	Scenario IDs, classes, expected behaviors, and truth labels
Rule library	Versioned deterministic rules and metadata
Similarity configuration	Methods, fields, weights, thresholds, and versions
Evaluation configurations	B0-B3 configuration files and declared primary profile
Processing and evaluation code	Commands or scripts used to generate all reported outputs
Expected outputs	Machine-readable reference result and human-readable summary
Audit-log sample	Examples of automatic, reviewed, rejected, unresolved, and contradiction outcomes
Tests	Unit or integration tests for generator, rules, accounting, and metric calculations
Checksums, license, and change log	Integrity, permitted use, and version history

9.1. Availability and archival record

A public website may introduce and link to the benchmark, but the exact release used for published results should be preserved in a version-controlled repository and, where practical, an archival repository with a stable identifier. The report must cite the exact tag or archived version rather than a moving branch.

10. Independent review and reproduction

Independent reproduction means that a technically qualified person who did not author the benchmark obtains the same main results using the provided artifacts. It is not the same as reading the report or endorsing the framework.

10.1. Reviewer procedure

1. Obtain the tagged or archived release without receiving unpublished files.
2. Follow the README in a clean environment and record any required clarification.
3. Generate or load the synthetic estate and verify declared checksums where applicable.
4. Execute the primary configuration and recompute the result file and tables.
5. Compare reproduced outputs with the published expected outputs and document exact matches or deviations.
6. Review lifecycle closure, selected error cases, and whether the report accurately describes the artifacts.
7. Issue a dated reproduction statement identifying the version, environment, procedure, findings, deviations, and limitations.

10.2. Permitted assistance and deviations

The author may answer clarification questions, but any required clarification or correction must be added to the public documentation or a new release. A reviewer should not silently patch code, alter data, change thresholds, or substitute unavailable dependencies. Deviations may be reported as exact, materially equivalent within a declared tolerance, or not reproduced.

10.3. Website presentation

The OGARD website may list the reviewer, review date, exact version, scope, summary finding, and a downloadable signed review or reproduction statement. It should not label the process as peer review or certification unless an external body actually performed such a process.

11. Results-reporting requirements and release gate

A future OGARD Synthetic Benchmark Report should be a separate document generated from the frozen artifacts and result files. It should report results even when they are weaker than expected and should never convert synthetic measurements into claims about operator performance.

11.1. Minimum results report

Section	Required content
Run identity	Release tag, commit, configuration ID, seed, environment, execution date, and reviewer status
Estate summary	Assets, records, source classes, scenario counts, and partition design
Configuration	Rules, similarity methods, thresholds, and baseline definitions
Results	Counts and metrics for B0-B3 with explicit denominators
Lifecycle accounting	Mutually exclusive record-disposition table proving closure
Error analysis	False accepts, missed matches, review outcomes, contradictions, and unresolved cases
Sensitivity	Declared threshold or configuration checks
Reproduction	Independent reviewer, environment, result comparison, and deviations
Limitations	Synthetic-data, scale, domain, and transferability limits

11.2. Release gate

- All claimed result artifacts exist and are included or linked from the exact release.
- Metric calculations and lifecycle totals reconcile from machine-readable files.
- The processing code does not read the ground-truth artifact.
- The README succeeds in a clean environment without undocumented intervention.
- The report agrees with the result files, configuration, and change log.
- Known failures and limitations are disclosed.
- Independent reproduction is completed before the website states that results were reproduced.

Release principle. The methodology does not impose a marketing-style minimum accuracy threshold. The release gate concerns honesty, accounting, reproducibility, and interpretability. Performance is reported, not manufactured.

12. Limitations and non-claims

12.1. Known derived-status evaluation gap and planned path

The v0.1 core benchmark evaluates identity reconciliation, review routing, contradiction detection, and auditability. It does not yet evaluate whether operator-approved derived-status rules produce the expected Available, Degraded, Offline, At risk, or Unknown outputs. This is a material validation gap because derived status is one of the framework's principal information products.

A later validation extension should freeze a small synthetic rule set, declare required evidence, freshness, expiry, contradiction overrides, and expected status outcomes, and test those rules against a separate evaluation set. The result would validate only the declared synthetic rules and evidence conditions; it would not establish physical asset truth or transfer automatically to operator-defined status semantics.

- Synthetic records cannot reproduce the full irregularity, history, access constraints, and organizational context of live utility systems.
- Scenario frequency is generator-controlled and should not be interpreted as sector prevalence.
- Thresholds suitable for a synthetic estate are not recommended operator thresholds.
- A benchmark can test the implemented reference logic only; it cannot validate all future OGARD implementations.

- No result establishes outage reduction, restoration-time improvement, production reliability, operator adoption, or economic impact.
- No result authorizes autonomous asset-status, diagnostic, maintenance, or operational-control decisions.
- Independent reproduction increases confidence that the artifacts produce the reported results; it is not a utility certification or live-domain validation.

References

[1] Bektas, D. OGARD: Open Grid Asset Reliability Data Framework. Draft Release Candidate v0.1, 2026.

[2] Bektas, D. OGARD Implementation Guide. Draft Release Candidate v0.1, 2026.

[3] Bektas, D. OGARD Architecture and Data Model Specification. Draft Release Candidate v0.1, 2026.

[4] Association for Computing Machinery, SIGSIM PADS. Reproducibility and Artifact Evaluation, 2026. Referenced for artifact qualities including documentation, consistency, completeness, exercisability, availability, reusability, and independent reproduction.

[5] National Institute of Standards and Technology. Reproducibility and Repeatability terminology. Referenced to distinguish same-conditions repeatability from results obtained by different experts or changed conditions.

Appendix A. Minimum configuration record

Field	Example form
release_version	ogard-v0.1
configuration_id	A unique, human-readable run identifier
repository_tag_or_commit	Exact immutable code reference
generator_version	Version or commit
random_seed	Integer
estate_size	Canonical asset count and source-record count
source_classes	Declared list
scenario_profile	Scenario counts or generator probabilities
partition_method	Calibration/evaluation split and seed
rule_library_version	Version or checksum
similarity_profile	Methods, fields, weights, and version
threshold_profile	Auto-accept, review, and reject bands
evaluation_version	Version or commit
environment	OS, runtime, dependency lock or container digest
execution_time	UTC timestamp and duration
model_profile_version	Exact OGARD model profile used by the generated and processed records
schema_bundle_version	Exact version or checksum of the machine-readable schema bundle

Appendix B. Results and disposition template

Record disposition	Count	Share	Notes
Input source/evidence records	[populate]	100%	All records generated or loaded
Invalid or quarantined	[populate]	[populate]	Reason categories required

Record disposition	Count	Share	Notes
Reconciliation-eligible	[populate]	[populate]	Input less invalid/quarantined
Automatically accepted - Tier 1	[populate]	[populate]	Correct and incorrect counts required
Automatically accepted - Tier 2	[populate]	[populate]	Correct and incorrect counts required
Rejected or no-match	[populate]	[populate]	Expected and unexpected rejections
Routed to review	[populate]	[populate]	Entry count, not a terminal disposition
Review accepted	[populate]	[populate]	Terminal review outcome
Review rejected	[populate]	[populate]	Terminal review outcome
Review unresolved	[populate]	[populate]	Active terminal-at-cutoff state
Lifecycle closure check	[pass/fail]		Mutually exclusive terminal totals reconcile

Appendix C. Artifact checklist

Item	Required before results publication
Methodology document	Final and consistent with the release
README and environment	Complete and tested from a clean environment
Generator or synthetic data	Included and versioned
Separate ground truth	Included or deterministically generated; not read by processing
Scenario catalogue	Included with expected behavior and labels
B0-B3 configurations	Included
Rules, similarity, and thresholds	Included and versioned
Processing and evaluation code	Included and exercisable
Machine-readable results	Included and matches report tables
Audit examples and tests	Included
Checksums, license, and change log	Included
Independent reproduction statement	Required before claiming reproduced results

Change log

Version	Date	Status	Summary
Draft RC v0.1	August 2026	Not publicly released	Initial methodology and validation plan aligned with the OGARD Whitepaper, Implementation Guide, and Architecture and Data Model Specification. Defines a controlled pre-results protocol, core entity and layer alignment, ground-truth separation, baselines, metrics, lifecycle closure, independent reproduction, and the results-release gate.
Draft RC v0.1 - Batch 1	August 2026	Not publicly released	Batch 1 review revision: added explicit model-profile and schema-bundle version requirements to benchmark configuration and reporting.
Draft RC v0.1 - Batch 2	August 2026	Not publicly released	Batch 2 review revision: documented scenario provenance, added an adversarial correlated-source blind spot, promoted review burden to a principal metric, and declared the derived-status evaluation gap with a future validation path.
Public edition v0.1.1	24 September 2026	Public	Status updated; content unchanged. Publication metadata, running headers and edition history updated. Added a front-matter pointer to the separate selected synthetic results first released in 0.2.0.